

Session 07 — Student Notes

TaskFlow: Lists + Derived Data

Quick recap — see the full project guide for deep explanations.

`.map()`

Turns an array into a list of React elements — one per item.

```
{visibleTasks.map((task) => {
  const statusText = task.completed ? "Completed" : "Pending";
  const statusClass = task.completed ? "completed" : "pending";

  return (
    <TaskItem
      key={task.id}
      title={task.title}
      ownerName={getOwnerName(task.userId)}
      statusText={statusText}
      statusClass={statusClass}
    />
  );
})}
```

Doesn't change the original array. Always give each item a stable `key` (`task.id` — unique, stable, already part of the data).

Arrow functions

A shorter function syntax: `function (x) { return x + 1; } → (x) => { return x + 1; } .`
Same behavior, less to type.

Derived data

If you can calculate a value from data + state you already have, calculate it — don't store it in another `useState` . `visibleTasks` , stats, counts, and "X of Y" are all derived, every render, never saved anywhere.

`.filter()` and `.reduce()` reused

Both already known — `.filter()` builds `visibleTasks`; `.reduce()` builds the dynamic stats. Same tools, now working on real derived data instead of a one-time calculation.

type + `FilterStatus`

```
type FilterStatus = "all" | "completed" | "pending";
```

Only these three strings are allowed for `currentFilter` — TypeScript rejects anything else.

Person filtering

`selectedUserId` (a number; `0` = "All people") is real state — it changes over time and React needs to know. "All people" is fully wired. The named person buttons render (with a real `key`) but don't do anything yet when clicked — that's homework.

Stats vs. "X of Y"

The three stat cards (Total/Completed/Pending) are derived from the *full* `tasks` array and never change with filtering. "X of Y tasks shown" is different — it reacts to whatever the current filters leave visible.

What's still not here

`tasks` is still a plain `const` array. Stateful task collection, CRUD, and callback props begin in Session 08. `useEffect` and API fetching come in Session 09. Homework finishes wiring the named person buttons and adds an explicit empty-result message — you write both yourself.